IndicDB - Benchmarking Multilingual Text-to-SQL Capabilities in Indian Languages

Anonymous authors
Paper under double-blind review

Abstract

While Large Language Models (LLMs) have significantly advanced Text-to-SQL performance, existing benchmarks predominantly focus on Western contexts and simplified schemas, leaving a critical gap in real-world, non-Western applications. We present IndicDB, a comprehensive multilingual Text-to-SQL benchmark designed to evaluate cross-lingual semantic parsing across diverse Indic language families. The foundational relational schemas for IndicDB are sourced from primary open-data platforms, specifically the National Data and Analytics Platform (NDAP, https://ndap.niti.gov.in/) and the India Data Portal (IDP, https://indiadataportal.com/), to ensure the benchmark accurately reflects the structural complexity of real-world administrative data. IndicDB comprises 20 databases across 237 tables. To transform denormalized government data into complex relational structures, we utilize an iterative three-agent judge pattern (Architect, Auditor, and Refiner) to ensure structural rigor and high relational density (11.85 tables per database; join-depths up to six). The methodology employs a value-aware, difficulty-calibrated, and join-enforced pipeline to systematically synthesize 15,612 tasks encompassing English, Hindi, and five primary Indic languages. We subsequently evaluate the cross-lingual semantic parsing performance of state-of-the-art models, including Deepseek v3.2, Multimax 2.7, Llama 3.3, and Qwen3, across seven linguistic variants to establish comprehensive performance baselines. Our results uncover a 7.18% global performance drop from English to Indic variants, highlighting a persistent "Indic Gap" driven by increased schema-linking difficulty, greater structural ambiguity in mapping Indic language to SQL, and lack of external knowledge. IndicDB serves as a rigorous "pressure test" for the cross-lingual Text-to-SQL synthesis and semantic parsing capabilities of large language models within linguistically diverse environments. The code and benchmark are publicly available at: https://anonymous.4open.science/r/multilingualText2Sql-Indic--DDCC/

1 Introduction

Text-to-SQL parsing aims to translate natural language questions into executable SQL queries, enabling non-expert users to interrogate relational databases without mastering query syntax. Driven by advances in Large Language Models (LLMs), performance on established benchmarks has improved dramatically: on Spider (Yu et al. (2018)), top-model execution accuracy rose from 53.5% to 85.3% in three years. The BIRD benchmark (Li et al. (2024)) raised the bar with 12,751 examples over 95 large, noisy databases (33.4 GB), yet even GPT-4 achieves only 54.89% - 38 points behind human performance. More recently, Spider 2.0 (Lei et al. (2025)) further expanded the scope to enterprise-grade data workflows spanning SQL, dialect diversity, and multi-turn interactions, reinforcing that real-world Text-to-SQL remains far from solved.

A critical blind spot in this progress, however, is its overwhelmingly **English-centric** nature. Spider, BIRD, Spider 2.0, and WikiSQL all use English-only schemas drawn from Western contexts. MultiSpider (Dou et al. (2023)) extends Spider to Chinese, Vietnamese, French, and Spanish, but inherits Spider's relatively simple, normalized schemas. Existing Text-to-SQL benchmarks predominantly focus on English-centric, simplified schemas that fail to encapsulate the administrative and linguistic complexities inherent to the **Global South**. IndicDB addresses this limitation by offering a specialized evaluation suite that rigorously tests the cross-lingual semantic parsing and structural reasoning capabilities of large language models across the diverse scripts and relational frameworks of the Indian subcontinent.

India's public data ecosystem, hosted on the **National Data and Analytics Platform (NDAP)**, **India Data Portal (IDP)**, and repositories such as **ICRISAT** and **IHDS**, offers a uniquely challenging testbed. These datasets feature deep administrative hierarchies (Country→State→District→Sub-District→Block→Village), yielding foreign-key chains of depth up to six. IndicDB incorporates domain-specific schemas that address thematic gaps in existing evaluation suites, featuring diverse sectors

LANGUAGE	AVG. PERF.	DROP
English	61.00%	—
Hinglish	56.98%	4.02%
Bengali	54.86%	6.14%
Hindi	54.49%	6.51%
Marathi	53.78%	7.22%
Tamil	53.00%	8.00%
Telugu	49.26%	**11.74**%

Table 1: Cross-lingual performance on IndicDB. The "Drop" column highlights the accuracy penalty relative to the English baseline. Observed performance gaps are primarily driven by: (1) Schema-linking difficulty; (2) Structural ambiguity; and (3) Reduced alignment with training patterns.

such as Household and Social Surveys, Census and Demography, among others. A health surveillance database may decompose into 18 tables covering routine immunization, maternal antenatal care, and family planning; an agricultural database may split output across Kharif and Rabi seasons with columns like `KHARIF_SORGHUM_YIELD_KG_PER_HA`. Combined with high-cardinality entity spaces (up to 569K unique geographic identifiers), these properties create extreme schema-linking demands, especially when queries arrive in Indic languages with zero lexical overlap to English-encoded column names.

1.1 IndicDB: Benchmark and Contributions

We present **IndicDB**, a large-scale multilingual Text-to-SQL benchmark grounded in real Indian administrative databases, evaluated across **seven linguistic variants**: English, Hinglish, Hindi, Bengali, Tamil, Telugu, and Marathi. Our contributions are:

- **Systematic Construction and Synthesis.** We curate **20 PostgreSQL databases (237 tables, 7.69M rows)** using a novel **three-agent judge pattern** (Architect, Auditor, Refiner) to produce complex star/snowflake schemas with join-depths up to six. This foundation supports **15,612 tasks** synthesized via a value-aware, join-enforced pipeline across seven languages, all rigorously verified by native-speaker experts.
- **Comprehensive Multi-Model Benchmarking.** We evaluate four state-of-the-art large language models—**Llama-3.3-70B**, **Qwen3-8B**, **MiniMax-M2.7**, and **DeepSeek-V3.2**—across one-shot, few-shot, and **DIN-SQL** prompting methodologies (Llama Team (2024); Qwen Team (2025); MiniMax AI (2026); DeepSeek AI (2025); Pourreza & Rafiei (2023)). prompting paradigms. Our framework specifically tests the impact of external knowledge augmentation (SEED (Yun & Lee (2025))) on cross-lingual grounding in high-cardinality environments.
- **Characterization of the "Indic Gap."** We uncover a consistent ∼**7.18%** global performance drop from English to Indic variantswith the most substantial deficit observed in Telugu, which exhibits a maximum decline of ∼**11.74%** , as detailed in **Table 1**. Through fine-grained **error analysis**, we categorize failure modes across schema complexities and linguistic nuances, providing actionable insights for improving multilingual Text-to-SQL reasoning.

2 Related Work

Cross-Domain and Enterprise-Scale Benchmarks. The progression of Text-to-SQL research has been defined by the evolution of benchmark complexity. **Spider** (Yu et al. (2018)) introduced the first large-scale, cross-domain dataset focusing on structural complexity. To address the limitations of clean, small-scale schemas, **BIRD** (Li et al. (2024)) shifted the focus toward massive database volumes and the necessity of external knowledge. Most recently, **Spider 2.0** (Lei et al. (2025)) has advanced the field toward enterprise-level requirements, incorporating real-world SQL dialects such as BigQuery and Snowflake alongside multi-turn agentic workflows. In contrast, **IndicDB** expands the frontier of structural and linguistic complexity by transforming massive, flat Indian government datasets into mathematically rigorous star and snowflake architectures via a novel 3-Agent Judge pipeline. Unlike existing datasets, our benchmark introduces unique structural challenges, such as multi-fact constellations and deep administrative geohierarchies, alongside the distinct entity-grounding challenges inherent to evaluating Text-to-SQL across six culturally rich Indic languages.

Multilingual and Cultural Grounding. To evaluate the cross-lingual capabilities of Large Language Models (LLMs), **MultiSpider** (Dou et al. (2023)) extended foundational Text-to-SQL tasks to seven

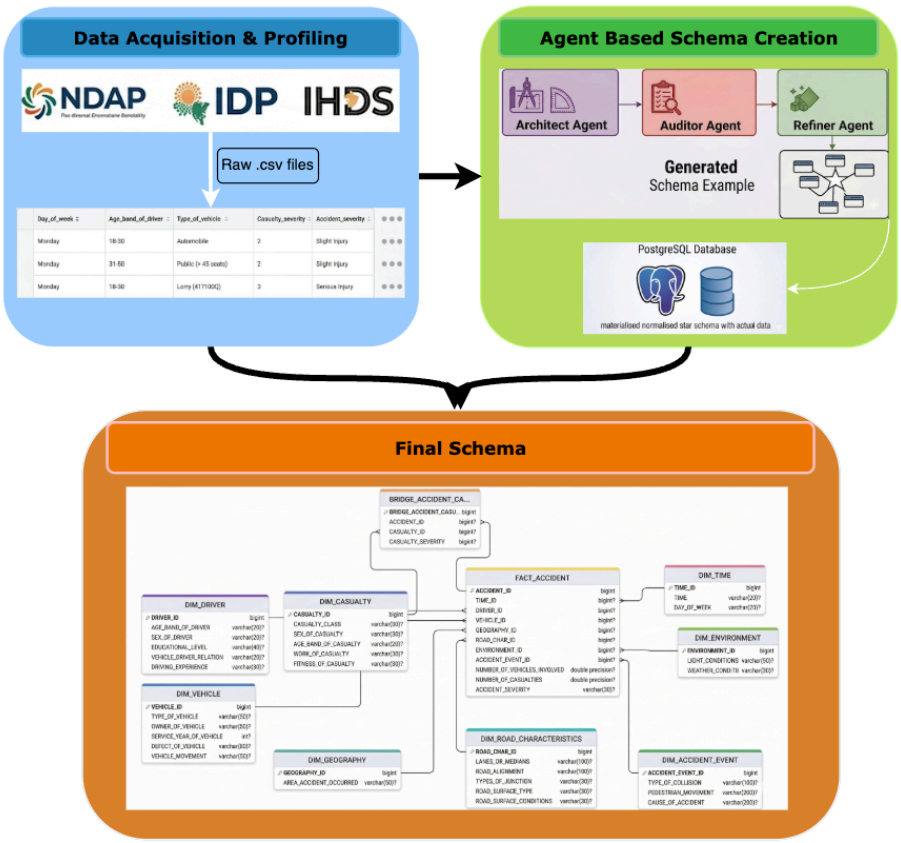

Figure 1: Schema creation pipeline

languages. This was further evolved in **MultiSpider 2.0** (Pham et al. (2025)), which applied enterprise-scale complexity to eight languages and identified a significant performance cliff for non-Western linguistic variants. In the broader Indian context, **IndicQA** (Singh et al. (2024)) established a high-bar for question answering across 11 major Indian languages, proving that models struggle with the morphological richness and script complexity of Indic variants. **IndicDB** bridges these domains by applying enterprise-grade relational density to authentic Indian context data.

Automated Task Generation. Benchmark synthesis has evolved from rule-based grammars to agentic LLM pipelines. Early benchmarks used recursive synchronous context-free grammars, which guaranteed structural correctness but produced limited linguistic and logical diversity. **DSQG-Syn** (Duan et al. (2025)) improved this by introducing difficulty-aware, question-guided SQL synthesis with iterative generation. **IndicDB** extends this line of work specifically for multilingual Text-to-SQL under realistic relational settings: we enforce schema-grounded join validity (FK-path-only joins) and increase hard-query coverage through controlled join/aggregation/CTE patterns.

3 The IndicDB Benchmark Construction

Building IndicDB proceeds in three phases: **[1] Schema Synthesis** - transforming flat government CSVs into rich relational structures, **[2] Task Generation** - synthesizing value-grounded Text-to-SQL tasks, and **[3] Multilingual Expansion** - producing faithful Indic language variants. We detail each below.

3.1 Agentic Schema Synthesis

Indian open-data sources (NDAP, India Data Portal) distribute datasets as monolithic CSVs with 50–100+ mixed-granularity columns. We convert these into complex relational structures via a **3-Agent Judge Pattern** (as shown in Figure 1) - an iterative, LLM-driven feedback loop:

- **Architect** facilitates the transition from monolithic datasets to structured relational environments by synthesizing normalized star or snowflake schemas. Within this architectural framework, high-dimensional source files are decomposed into a collection of four to ten thematic entities categorized as either **Fact Tables** (identified by the *FACT_* prefix) or **Dimension Tables** (identified by the *DIM_* prefix), maintaining a strict limitation of fifteen columns per table. The design centralizes quantita-

Table 2: IndicDB Comprehensive Framework: Qualitative Complexity Taxonomy (Left) and Full Quantitative Benchmark Statistics (Right).

Feature / Constraint	Easy	Medium	Hard
Relational Depth	0–1 JOIN	Exactly 1 JOIN	$\geq$ 2 JOINs
JOIN Diversity	INNER JOIN only	INNER JOIN primarily	Diverse (INNER, LEFT, RIGHT)
Filtering Logic	Simple `WHERE`	Moderate (e.g., Ranges)	Complex multi-column filters
Aggregation	None	$\leq$ 1 Clause	Required (GROUP BY)
Nesting	Prohibited	Prohibited	Required (CTEs/Sub-Q)
SQL Tokens	$<$ 60	60–120	$>$ 120

Category	Metric	Count	Pct. / Avg.
Volume	Total Size	15,617	–
	Unique Pairs	3,684	–
Language	English	3,684	30.1 w
	Hindi	1,948	33.0 w
	Indic-4*	8,248	24.1 w
	Hinglish	1,737	29.0 w
Difficulty	Easy	1,055	28.6%
	Medium	1,539	41.8%
	Hard	1,085	29.5%
SQL Op.	JOIN	3,484	94.6%
	WHERE	3,278	89.0%
	GROUP BY	2,441	66.3%
	ORDER BY	2,289	62.1%
Agg.	COUNT()	929	25.2%
	SUM()	809	22.0%
	AVG()	560	15.2%

*Indic-4 includes: Marathi, Bengali, Tamil, and Telugu.

tive metrics and foreign key identifiers within a primary Fact Table (e.g., `Fact_Accident_Occurrences`, which records the raw count of crashes, fatalities, and injuries per incident), which is subsequently linked to surrounding Dimension Tables that provide granular context through attributes such as temporal or geographic metadata (e.g., `Dim_Time_Periods` for identifying the specific month and year, and `Dim_Geographic_Regions` for detailing the state, district, and weather conditions). By enforcing this structural separation, the pipeline ensures that semantic parsing models must navigate complex multi-hop join operations and demonstrate precise schema-linking to achieve accurate query synthesis.

- **Auditor** evaluates the proposed architecture against rigorous design constraints. It audits the schema for Third Normal Form (3NF) violations, poor thematic cohesion, and table width violations. Beyond basic normalization, it assesses the relational graph's complexity, ensuring that the primary-to-foreign key linkages are deeply nested enough to guarantee that analytical queries will require advanced, multi-hop joins ($\geq$3 tables).
- **Refiner** employs an **LLM-as-a-judge** paradigm to adjudicate between the Architect's proposed structure and the Auditor's critical feedback, facilitating the synthesis of a **finalized schema**. This component systematically resolves identified structural flaws, standardizes heterogeneous or ambiguous column headers into canonical SQL identifiers, and enforces strict data typing across all fields. By leveraging the evaluative capabilities of the LLM to weight the auditor's corrections against the initial draft, the Refiner generates a formalized schema configuration file that maintains a mathematically precise, **one-to-one mapping** back to the underlying denormalized source data.

The agentic output was compiled directly into a Data Definition Language (DDL) file, establishing foreign key relationships and surrogate keys for hierarchical administrative data (Country → State → District → Sub-District → Village). Following rigorous manual verification by a team of database experts, this DDL file was executed and the final dataset was bulk-loaded into PostgreSQL.

3.2 Task Synthesis via Enhanced DSQG-Syn

We adopt the DSQG-Syn framework (Duan et al. (2025)) for its **Question-First** paradigm: rather than randomly sampling columns to construct SQL (which often produces intent-inconsistent pairs), it first generates domain-relevant questions across nine predefined types covering all major SQL operations (Scan, Aggregate, Filter, Sort, TopSort, Join, Except, Intersect, Union), then synthesizes grounded SQL-NLQ pairs.

Our enhanced pipeline operates in four stages per database:

1. **Question Generation.** A schema graph is constructed from FK relationships; BFS selects connected table subsets. Domain keywords are extracted via LLM, and nine question types are generated per table group.
2. **Schema Linking.** A MAC-SQL–inspired selector identifies the minimal relevant sub-schema for each question, augmented with sample values from PostgreSQL.

Language	Question
English	Which districts in Chhattisgarh achieved a rice yield exceeding 828.55 kg per hectare during the year 1970?
Hindi	छत्तीसगढ़ के कौन से जिलों ने वर्ष 1970 के दौरान प्रति हेक्टेयर 828.55 किलोग्राम से अधिक धान की पैदावार हासिल की?
Bengali	ছত্তিশগড়ের কোন জেলাগুলি ১৯৭০ সালে প্রতি হেক্টরে ৮২৮.৫৫ কেজির বেশি ধান উৎপাদন অর্জন করেছিল?
Marathi	छत्तीसगढमध्ये १९७० साली कोणत्या जिल्ह्यांनी प्रति हेक्टर ८२८.५५ किलोग्रामपेक्षा जास्त भात उत्पन्न मिळवले?
Tamil	சத்தீஸ்கரில் எந்த மாவட்டங்கள் 1970 ஆம் ஆண்டில் ஒரு ஹெக்டேருக்கு 828.55 கிலோகிராமுக்கும் அதிகமான நெல் மகசூலை அடைந்தன?
Telugu	ఛత్తీస్‌గఢ్‌లో 1970 సంవత్సరంలో ఏ జిల్లాలు హెక్టారుకు 828.55 కిలోలకు మించిన బియ్యం దిగుబడిని సాధించాయి?
Hinglish	Chhattisgarh mein kaunse districts ne saal 1970 mein 828.55 kg per hectare se zyada rice yield haasil kiya?

```
SELECT T1.DIST_NAME FROM DIM_GEOGRAPHY AS T1 JOIN
FACT_CEREALS_MAJOR AS T2 ON T1.GEOGRAPHY_ID =
T2.GEOGRAPHY_ID JOIN DIM_YEAR AS T3 ON T2.YEAR_ID =
T3.YEAR_ID WHERE T1.STATE_NAME = 'Chhattisgarh' AND
T2.RICE_YIELD_KG_PER_HA > 828.55 AND T3.YEAR = '1970'
```

Figure 2: Example of a generated multilingual task

3. **Skeleton-Guided SQL Generation.** Abstract SQL templates with placeholders are generated at three difficulty tiers (Easy 30% / Medium 40% / Hard 30%) as defined in Table 2, then filled with actual schema names and *real database values*, eliminating "predicate hallucination" where models fabricate filter values.
4. **NLQ Synthesis.** During translation, we strictly enforced "linguistic vagueness" to ensure the generated Natural Language Questions (NLQs) sounded like real human queries rather than robotic, literal translations of the SQL code. By intentionally hiding exact database table and column names (e.g., asking "*How many private clinics are there?*" instead of "*Count the hospital IDs in the* `dim_facilities` *table where the type is Private*"), we force the underlying semantic parser to genuinely understand the domain rather than simply relying on basic keyword matching.

FK-Constrained Join Enforcement - A critical enhancement is constraining the SQL generator to follow *only* declared FK paths. Without this, LLMs frequently perform semantically invalid joins by matching columns with similar names but distinct semantics (e.g., joining `STATE_ID` with `STATION_ID`). We inject allowed FK relationships into the generation prompt and apply a type-safety filter that drops SQL applying numeric operations to non-numeric columns.

Task Statistics - The final English set comprises **3,684 validated NLQ-SQL pairs**, following a calibrated difficulty mix. All tasks were validated via SQL execution against PostgreSQL and a random subset was further verified through human expert audit.

3.3 Multilingual Expansion

We expand English tasks into six additional variants: **Hindi, Bengali, Tamil, Telugu, Marathi**, and **Hinglish** (HI-EN code-switching), yielding **15,617 total tasks**. We adopt an English-First approach: only the NLQ is translated while the SQL remains identical, ensuring perfect logical alignment across variants. Gemini 3 Flash serves as the primary conversion engine.

Hinglish receives specialized prompting for *Natural Hinglish*—Roman script blending Hindi grammar with English technical terms (e.g., "*Agriculture department mein kitne records hain?*")—testing model performance on high-usage but low-resource linguistic patterns.

Quality Assurance and Verification – We implement a **multi-stage Human-in-the-Loop (HITL)** verification framework to ensure cross-lingual semantic equivalence. This process centers on a comprehensive **back-translation phase** where Indic natural language queries are mapped back to English to identify semantic drift or structural misalignments. All identified discrepancies were **manually corrected** (Examples shown in **Table 5**) in by native-speaking experts to guarantee that each localized instance remains a faithful representation of the gold-standard SQL intent.

Our systematic audit of the benchmark revealed two predominant categories of recurrent errors that were addressed during the manual refinement phase:

- **Lexical Entity Divergence:** The mistranslation of domain-specific entities—such as agricultural crop varieties—which directly compromised the precision of SQL `WHERE` clause filters.
- **Logical Directional Inversion:** Instances where sorting directives were erroneously swapped in the target script, requiring manual rectification of the corresponding `ORDER BY` logic.

These targeted interventions, integrated with automated **smoke tests** on localized database snapshots, ensure consistent **schema grounding** and functional integrity across the IndicDB framework.

4 Experiments

4.1 Experimental Setup

Language Selection. We evaluate the robustness of text-to-SQL systems across a linguistically diverse set of Indic and code-mixed settings. Our study encompasses seven languages: English, which serves as the baseline; five typologically diverse Indic languages - Hindi, Marathi, Bengali, Tamil, and Telugu; and Hinglish, a code-mixed Hindi–English variant that reflects real-world usage in multilingual contexts.

To ensure a controlled comparison across languages, we keep the underlying database schema fixed and vary only the natural language queries via translation.

Models We evaluate a diverse set of recent large language models that span a range of architectural designs and model scales. Our evaluation includes Llama 3.3 70B Instruct (70B parameters) (Llama Team, 2024), Qwen3 8B (8B parameters) (Qwen Team, 2025), decoder-only transformers; DeepSeek V3.2 (DeepSeek AI, 2025), a mixture-of-experts transformer with a total parameter count exceeding 671B (with a smaller subset activated per token); and MiniMax M2.7 (MiniMax AI, 2026), a recent large language model with agent-oriented capabilities and self-evolving training mechanisms..

All models are used off-the-shelf without any task-specific fine-tuning.

Prompting Strategies

We evaluate model performance under two prompting strategies: one-shot prompting and the DIN-SQL (Pourreza & Rafiei, 2023) framework. DIN-SQL decomposes text-to-SQL generation into a sequence of structured intermediate steps, including schema linking, clause-wise SQL construction, and iterative self-correction, which together improve reasoning and execution accuracy. We further augment DIN-SQL with knowledge files to provide explicit grounding signals during generation.

For one-shot prompting, we consider two settings: with and without auxiliary knowledge files. The knowledge files are constructed using the SEED (Yun & Lee, 2025) approach and provided as plain text. They contain schema linking cues, relevant column values, and SQL generation hints such as JOIN conditions, aggregation functions (e.g., SUM), and task-specific formulae. For DIN-SQL, we evaluate the knowledge-augmented variant across all languages and conduct ablation studies by removing knowledge files to assess their impact.

Mistranslation Examples	Fix
Q_en: List the names of districts that produced maize but did not produce any wheat during the year 1970, sorted alphabetically. **Q_ma (bad):** १९७० साली ज्वारी पिकवलेले पण गहू पिकवले नसलेल्या जिल्ह्यांची नावे वर्णक्रमानुसार सूचीबद्ध करा. **Q_ma (fix):** १९७० साली मका पिकवलेले पण गहू पिकवले नसलेल्या जिल्ह्यांची नावे वर्णक्रमानुसार सूचीबद्ध करा.	Corrected the **lexical mistranslation** by replacing ज्वारी (sorghum) with मका (maize). Using the wrong crop alters the query semantics and can lead to incorrect filtering in SQL generation.
Q_en: List the districts in India for the 1991 census year, ordered by the number of male workers in trade and commerce in descending order, and show only the top 10 results. **Q_bn (bad):** ১৯৯১ সালের আদমশুমারি অনুযায়ী, বাণিজ্য ও ব্যবসায়ে পুরুষ শ্রমিক সংখ্যার উপর ভিত্তি করে উর্ধ্বক্রমে সাজানো ভারতের জেলাগুলির তালিকা দিন এবং শুধুমাত্র শীর্ষ ১০টি ফলাফল দেখান। **Q_bn (fix):** ১৯৯১ সালের আদমশুমারি অনুযায়ী, বাণিজ্য ও ব্যবসায়ে পুরুষ শ্রমিক সংখ্যার উপর ভিত্তি করে অবরোহ ক্রমে সাজানো ভারতের জেলাগুলির তালিকা দিন এবং শুধুমাত্র শীর্ষ ১০টি ফলাফল দেখান।	Corrected the **ordering direction** by replacing উর্ধ্বক্রমে (ascending order) with অবরোহ ক্রমে (descending order) to match the intended sorting in the query.

Table 3: Mistranslation examples and corresponding fixes

Method	Model	English	Hindi	Bengali	Marathi	Tamil	Telugu	Hinglish
DIN-SQL (w/ evidence)	LLaMA 3.3 70B Instruct	66.10%	57.97%	68.61%	54.52%	58.09%	57.98%	65.07%
	Qwen3 8B	55.05%	52.65%	51.14%	49.36%	49.38%	49.98%	51.06%
	DeepSeek V3.2	69.07%	64.45%	65.36%	63.60%	66.67%	63.61%	66.17%
	Minimax 2.7	59.29%	66.66%	62.85%	75.00%	67.64%	-	80.30%
One-shot (w/ evidence)	LLaMA 3.3 70B Instruct	58.06%	44.24%	45.14%	42.20%	42.30%	39.46%	43.13%
	Qwen3 8B	52.17%	38.39%	37.52%	36.40%	34.23%	34.50%	38.58%
	DeepSeek V3.2	69.32%	57.66%	58.06%	56.66%	56.04%	52.94%	60.53%
	Minimax 2.7	59.51%	53.30%	50.95%	52.00%	49.39%	48.23%	50.21%
One-shot (w/o evidence)	LLaMA 3.3 70B Instruct							
	Qwen3 8B							
	DeepSeek V3.2							
	Minimax 2.7							
Max Performance Drop		0.00%	-13.82%	-14.65%	-15.86%	-17.94%	-18.60%	-14.93%

Table 4: Execution accuracy (EA) across languages for different prompting strategies. The final row reports the maximum performance drop relative to English.

We use identical prompt templates across languages and keep the number of in-context examples fixed to ensure fair comparison. All experiments use deterministic decoding (temperature = 0, top-p = 1). We adopt DIN-SQL due to its structured decomposition, which improves schema grounding and compositional reasoning. The addition of knowledge files further strengthens alignment between natural language queries and database structure, leading to consistent gains in execution accuracy, particularly in multilingual settings where lexical variation is high.

Evaluation metrics We evaluate performance using Execution Accuracy (EX) (Yu et al., 2018; Li et al., 2024), which measures whether the predicted SQL query produces the same result as the ground truth when executed on the database. For each example (j), let $\hat{S}_j$ denote the predicted query and S_j^* denote the corresponding gold query. The metric is defined as:

$$E_X = \frac{1}{m} \sum_{j=1}^{m} \mathbf{1} \left[\text{Exec}(\hat{S}_j) = \text{Exec}(S_j^*) \right]$$

where Exec$(\cdot)$ returns the result set from executing the query on the database.

4.2 Experimental Results and Analysis

4.2.1 Main Results

We present the main results across languages in Table 4. Across all 13,432 tasks and seven linguistic variants, we observe a global average performance drop of 7.18% relative to English, indicating a consistent cross-lingual degradation in text-to-SQL performance.

To provide a method-agnostic view, we compute the average accuracy across both prompting strategies and all models for each language. Hindi and Bengali exhibit moderate degradation (-6.66% and -6.12%, respectively), while Marathi and Tamil show slightly larger drops of -7.35% and -8.10%. Telugu exhibits the largest drop at -11.80%, whereas Hinglish shows the smallest drop (-4.19%) and achieves performance closest to English.

These results indicate that multilingual performance varies significantly across languages, with consistent degradation observed relative to English.

4.2.2 Error Analysis

To better understand the causes of multilingual performance degradation, we analyze model errors across all languages and focus on the two dominant categories: schema linking errors and aggregation/group-by errors, which together account for the majority of failures (See Figure 3).

Schema linking errors (20%) arise from incorrect alignment between natural language mentions and database schema elements, including tables, columns, and entities. These errors are particularly pronounced in languages such as Telugu, which exhibit the largest performance drop in our evaluation. The increased linguistic distance from English, combined with morphological variation and script differences, often leads to greater divergence between query tokens and schema representations, making accurate grounding more difficult. In contrast, Hinglish shows the smallest performance drop and correspondingly fewer schema linking errors. The presence of English tokens within code-mixed queries

appears to facilitate more direct alignment with schema elements, reducing ambiguity and improving grounding.

Aggregation and group-by errors (28%) represent the largest category of structural mistakes, primarily involving missing or incomplete GROUP BY clauses and incorrect aggregation behavior. These errors reflect limitations in compositional reasoning, where models fail to correctly infer aggregation constraints from the query. This challenge is amplified in multilingual settings, where variations in how quantitative or comparative intent is expressed can obscure the underlying structure of the query. As a result, models often capture the relevant entities but fail to construct the correct SQL operations.

Taken together, these findings suggest that multilingual text-to-SQL failures are driven by two key factors: (1) lexical misalignment between query and schema, and (2) difficulty in reasoning over compositional query structure. The contrast between Telugu and Hinglish further highlights that both linguistic distance and the degree of overlap with English play a critical role in determining model performance.

4.2.3 Case Studies

To further analyze model behavior in multilingual settings, we present representative case studies in Figure 4, highlighting common failure modes across languages. These examples demonstrate that even when translations are semantically correct, models often fail to map natural language queries to the appropriate SQL structure.

In the first case (Hindi), the model exhibits a Missing GROUP BY clause error. The query specifies conditions over aggregated quantities (e.g., total number of houses and population), which require grouping and filtering via `GROUP BY` and `HAVING`. However, the predicted SQL instead applies row-level `WHERE` filters and omits aggregation entirely. This indicates a failure to correctly interpret aggregation cues and map them to compositional SQL constructs.

In the second case (Telugu), the model produces a Wrong Logical Operator error. The natural language query expresses a disjunctive condition ("either ... or ..."), but the predicted SQL incorrectly replaces the intended `OR` operator with `AND`. This results in overly restrictive filtering and demonstrates the model's difficulty in preserving logical semantics under multilingual variations.

In the third case (Bengali), the model makes an Incorrect JOIN key error. While the relevant tables are correctly identified, the join condition is incorrectly specified, leading to a mismatch in schema relationships. This reflects a failure in schema linking, where the model is unable to accurately align natural language references with the underlying database structure.

Across these examples, a consistent pattern emerges: errors are primarily driven by failures in structural reasoning and schema alignment, rather than incorrect translation. Multilingual queries often express the same intent using more diverse linguistic constructions, which weakens the explicit cues required for mapping to SQL operators and schema elements. As a result, models frequently default to simpler or incorrect structural interpretations, leading to systematic errors in aggregation, logical reasoning, and join conditions.

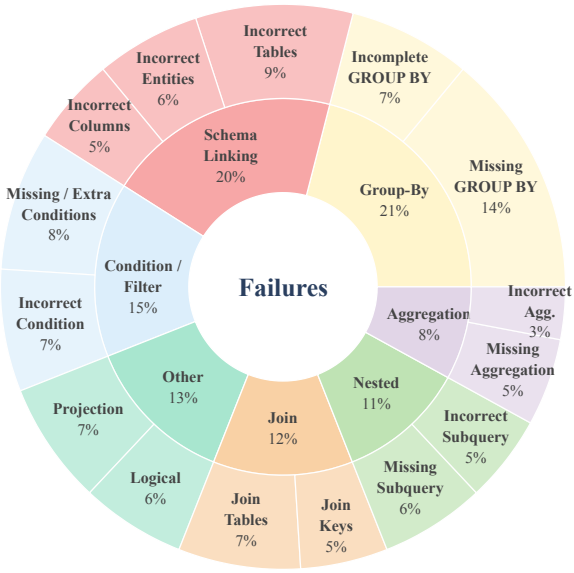

Figure 3: Distribution of error categories on the benchmark.

4.2.4 *Ablation study - Knowledge Files*

We evaluate the impact of knowledge files using DeepSeek V3.2 across 400 tasks spanning seven languages. The knowledge files consist of structured signals, including schema linking information, column value mappings, and SQL generation hints (e.g., joins, aggregations, and filtering patterns), injected into the prompt to guide query generation (See Figure **??**). As shown in Table 6, incorporating knowledge files leads to consistent improvements across all languages, with an average increase of approximately **+24–27%** in execution accuracy (EA).

To understand these gains, we analyze cases where the baseline fails but the knowledge-augmented query succeeds. As shown in Figure 5, most improvements arise from grounding-related corrections, particularly in selecting the correct schema elements and mapping natural language expressions to canonical database values. Many baseline errors correspond to result mismatches, where the SQL is executable but semantically incorrect due to misaligned columns, filters, or values. Knowledge files address these issues by explicitly linking query phrases to schema components and clarifying value representations.

In addition to grounding, knowledge files improve query structure and compositional reasoning. The model is better able to recover missing aggregation logic (e.g., `GROUP BY`, `HAVING`), correct join conditions, and construct accurate filters. This suggests that knowledge signals act not only as a mapping mechanism but also as a lightweight scaffold for SQL construction.

The impact varies across languages, with larger gains observed in lower-performing settings. Marathi shows the highest improvement ($\approx +27.5\%$), followed by Tamil ($\approx +27.3\%$) and Telugu ($\approx +25.7\%$), reflecting their weaker baselines and greater reliance on explicit grounding. In contrast, English exhibits smaller gains ($\approx +23.7\%$), likely due to its stronger baseline and closer lexical alignment with schema elements.

Overall, the gains from knowledge files are driven by improved semantic grounding and structural alignment rather than surface-level SQL refinement, with the largest benefits observed where the gap between natural language and schema representations is most pronounced.

Case Studies	
Question (Hindi)	1991 की जनगणना के आंकड़ों के आधार पर, उन स्थानों के नाम सूचीबद्ध करें जहाँ कुल आवासित आवासों की संख्या 5000 से अधिक थी और कुल महिला अनुसूचित जनजाति जनसंख्या 1000 से अधिक थी, तथा परिणामों को स्थान के नाम के अनुसार वर्णमाला क्रम में व्यवस्थित करें। (Based on the 1991 census data, list the names of places where the total number of occupied houses exceeded 5000 and the total female Scheduled Tribe population exceeded 1000, and arrange the results in alphabetical order by place name.)
GOLD SQL	SELECT (...) GROUP BY T1.location_name HAVING SUM(T2.number_of_occupied_residential_houses) > 5000 AND SUM(T3.female_scheduled_tribe_population) > 1000 ORDER BY T1.location_name
Predicted SQL	SELECT (...) WHERE T2.number_of_occupied_residential_houses > 5000 AND T3.female_scheduled_tribe_population > 1000 AND T2.census_year = 1991 AND T3.census_year = 1991 ORDER BY T1.location_name
Error Reason	Missing GROUP BY clause
Question (Telugu)	2022 సంవత్సరానికి చెందిన లేదా 5,000 కంటే ఎక్కువ అలోపతి బయటి రోగుల సందర్శనలను నమోదు చేసిన నెలలను గుర్తించండి, వర్ణమాల క్రమంలో క్రమబద్ధీకరించబడింది. (Identify the months that belong to the year 2022 or recorded more than 5,000 allopathy outpatient visits, and order the results alphabetically.)
GOLD SQL	SELECT (...) WHERE T2.year_value = 2022 OR T3.outpatient_attendance_allopathic > 5000 ORDER BY T1.month_name
Predicted SQL	SELECT (...) WHERE T2.year_value = 2022 AND T3.outpatient_attendance_allopathic > 5000 ORDER BY T1.month_name
Error Reason	Wrong Logical Operator
Question (Bengali)	এই প্রশ্নটি অনুবাদ করুন: ২০২১ সালে রেকর্ড করা প্রথম ডোজ ভিটামিন এ প্রশাসনের সর্বোচ্চ মোট সংখ্যা সহ আন্দামান ও নিকোবর দ্বীপপুঞ্জের শীর্ষ পাঁচটি উপজেলার তালিকা দিন। (List the top five sub-districts in the Andaman and Nicobar Islands with the highest total number of first-dose Vitamin A administrations recorded in 2021.)
GOLD	SELECT (...) JOIN fact_child_immunization_specialized AS T3 ON T1.geography_key = T3.geography_key JOIN dim_time AS T2 ON T3.time_key = T2.time_key (...)
Predicted SQL	SELECT (...) JOIN FACT_CHILD_IMMUNIZATION_SPECIALIZED AS T2 ON T1.GEOGRAPHY_ID = T2.GEOGRAPHY_ID JOIN DIM_TIME AS T3 ON T2.TIME_ID = T3.TIME_ID (...)
Error Reason	Incorrect JOIN key

Figure 4: Case studies illustrating lexical and structural errors across languages.

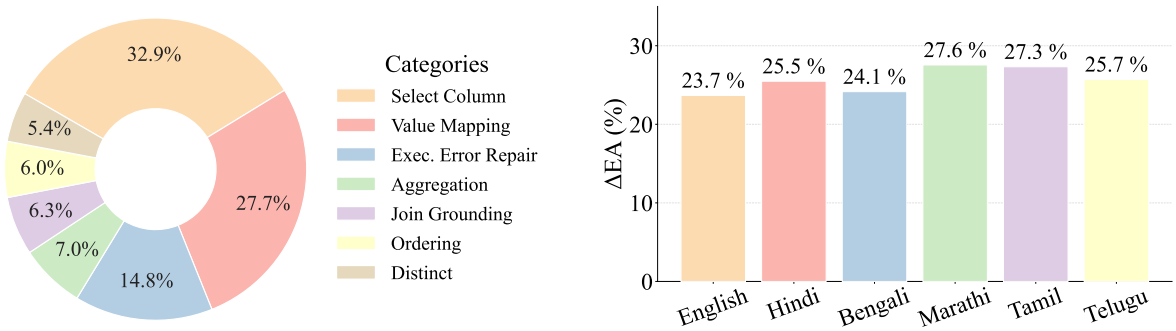

Figure 5: Impact of knowledge files: (Left) distribution of improvements, (Right) execution accuracy gains across languages.

5 Limitations and Future Directions

6 Conclusion

Acknowledgments

This paper is supported by NSF of China (62402409), Guangdong provincial project 2023CX10X008, Guangdong Basic and Applied Basic Research Foundation (2023A1515110545), Guangzhou Basic and Applied Basic Research Foundation (2025A04J3935), HKUST(GZ) Red Bird M.Phil. Program, and Guangzhou-HKUST(GZ) Joint Funding Program (2025A03J3714).

References

DeepSeek AI. Deepseek-v3.2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.

Longxu Dou, Yan Gao, Mingyang Pan, Dingzirui Wang, Wanxiang Che, Dechen Zhan, and Jian-Guang Lou. Multispider: towards benchmarking multilingual text-to-sql semantic parsing. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 12745–12753, 2023.

Shaoming Duan, Youxuan Wu, Chuanyi Liu, Yuhao Zhang, Zirui Wang, Peiyi Han, Shengyuan Yu, Liang Yan, and Yingwei Liang. DSQG-syn: Synthesizing high-quality data for text-to-SQL parsing by domain specific question generation. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Findings of the Association for Computational Linguistics: NAACL 2025*, pp. 2971–2989, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.162. URL https://aclanthology.org/2025.findings-naacl.162/.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, et al. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. In *The Thirteenth International Conference on Learning Representations* (*ICLR*), 2025. URL https://openreview.net/forum?id=XmProj9cPs.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sql. In *Advances in Neural Information Processing Systems* (*NeurIPS*), 2024.

AI at Meta Llama Team. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Question – Evidence pairs	
Question	Provide the area, production, and yield statistics for maize and barley in Chhattisgarh for the year 1970.
Evidence	Select maize and barley area, production, yield from fact_cereals_minor where dim_geography.state_name = 'Chhattisgarh' and dim_year.year = 1970.
Question	List the station code and the type of water body for all stations located in the state of Assam.
Evidence	Assam is a value in dim_state.state; join dim_station with dim_state on state_id; select station_code and type_of_water_body.

Table 5: Question–Evidence pairs for Text-to-SQL reasoning

Setting	English	Hindi	Bengali	Marathi	Tamil	Telugu
Without Knowledge	45.00%	39.75%	40.61%	36.75%	39.75%	37.90%
With Knowledge	69.07%	64.45%	65.36%	63.60%	66.67%	63.61%
Δ (**Gain**)	+24.07%	+24.70%	+24.75%	+26.85%	+26.92%	+25.71%

Table 6: Execution accuracy (EA) with and without knowledge file augmentation across languages.

MiniMax AI. Minimax m2.7: Early echoes of self-evolution. https://www.minimax.io/news/minimax-m27-en, 2026. Accessed: 2026.

Khanh Trinh Pham, Thu Huong Nguyen, Jun Jo, Quoc Viet Hung Nguyen, and Thanh Tam Nguyen. Multilingual text-to-sql: Benchmarking the limits of language models with collaborative language agents. In *Australasian Database Conference*, pp. 108–123. Springer, 2025.

Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *arXiv preprint arXiv:2304.11015*, 2023.

Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Harman Singh, Nitish Gupta, Shikhar Bharadwaj, Dinesh Tewari, and Partha Talukdar. Indicgenbench: A multilingual benchmark to evaluate generation capabilities of llms on indic languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics* (*Volume 1: Long Papers*), pp. 11047–11073, 2024.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (*EMNLP*), pp. 3911–3921, 2018.

Janghyeon Yun and Sang-goo Lee. Seed: Enhancing text-to-sql performance and practical usability through automatic evidence generation. In *Proceedings of the IEEE ICDE Workshops* (*ICDEW*), 2025.

A Appendix

A.1 Prompt Template

A.1.1 Prompt for Schema Linking

Schema Linking Prompt

You are a smart and responsible SQLite SQL expert. Assist in identifying the database tables and columns involved in natural language queries.

Instruction:
Your task is to analyze the provided database schema, comprehend the posed question, and leverage the hint to identify which tables are needed to generate a SQL query for answering the question. The returned JSON format must strictly adhere to the following specifications:

```
{
    "tables": [
        {
            "table": "table name",
            "columns": ["relevant column 1", "relevant column 2", ...]
        },
        ...
    ]
}
```

Each relevant column must belong to its respective table, and the output JSON object must be wrapped in a code block using ```json```. Please note that each table and column comes with detailed description information and example values for reference.

Database schema:
{schema_str}

User question:
{query}

Hint: {evidence}

A.1.2 Prompt for Basic SQL *Generation Pipeline*

Basic SQL Generation Pipeline

You are a intelligent and responsible SQLite expert.

Instruction:
You need to read the database schema to generate SQL query for the user question.
The outputted SQL must be surrounded by ```sql``` code block.

Database Schema:
{schema}

Hint:
{evidence}

User Question:
{query}

The outputted SQL must be surrounded by ```sql``` code block.

A.1.3 Prompt for Divide-and-Conquer Chain-of-Thought

Divide Prompt

You are a smart and responsible SQLite SQL expert. Given a database schema and a question, users want to know the corresponding SQL query. Your task is to understand the database schema and question, and decompose the question into sub-questions so user can better understand it. Each sub-question is enclosed in «». Here is an example for reference:

Example:

Given the database schema:
{example_database_schema}

Question:
{example_question}

Decompose the Question into sub-questions, each sub-question is enclosed in «»:
Sub-question 1: «{sub question 1}»
Sub-question 2: «{sub question 2}»
Sub-question 3: «{sub question 3}»

Your task: decompose the question into sub-questions.

Given the database schema:
{schema}

Question:
{query}

Hint:
{evidence}

Decompose the Question into sub-questions, each sub-question is enclosed in «»:

Conquer Prompt

You are a smart and responsible SQLite SQL expert. Given a database schema and a question, your tasks are:

1. Parse user questions: Use natural language processing (NLP) techniques to parse user questions and extract query requirements and conditions.

2. Analyze database schema: Based on the database schema, understand the fields and relationships of the table, and build the basic framework of the SQL query.

3. Check sample data: Analyze the data characteristics based on the first three rows of the table values to help determine how to construct query conditions and filter results.

4. Generate SQL query: Based on user questions, query requirements and conditions, database schema, and sample data, build a complete SQL query.

5. Verification and optimization: Check whether the generated SQL query is logical and optimize it if necessary.

Database Schema:
{schema}

Examples:
{examples}

Question:
{query}

Hint:
{evidence}

Please generate the corresponding SQL query. SQL must be surrounded by ```sql``` code block.

Assemble Prompt

You are a smart and responsible SQLite SQL expert. Given a database schema and a question, users want to know the corresponding SQL query.

Instructions:
We have decomposed the main question into sub-questions, now your task is based on the SQL querys for corresponding sub-questions, assemble the final SQL for the main question:
1. Understand the database schema and the main question;
2. Read and analyze each sub-question and corresponding SQL query;
3. Analyze the relationship between sub-questions and the main question in order to assemble them properly;
4. Generate the final SQL for the main question and optimize it if needed.

Database Schema:
{schema}

Main question:
{query}

Hint:
{evidence}

Sub-questions and corresponding output, including SQL querys and explanation:
{subs}

Based on the SQL querys for corresponding sub-questions, generate the final SQL for the main question in the end of your response, SQL must be surrounded by ```sql``` code block.

A.1.4 DIN-SQL Prompt

Debugger Prompt

For the given question, use the provided tables, columns, foreign keys, and primary keys to fix the given SQLite SQL QUERY for any issues. If there are any problems, fix them. If there are no issues, return the SQLite SQL QUERY as is.

Use the following instructions for fixing the SQL QUERY:

1) Pay attention to the columns that are used for the JOIN by using the Foreign_keys.

2) Use DESC and DISTINCT when needed

3) Pay attention to the columns that are used for the GROUP BY statement

4) Pay attention to the columns that are used for the SELECT statement.

5) Only change the GROUP BY clause when necessary (Avoid redundant columns in GROUP BY).

6) The question may be in non-english language, the sql query has to be in english.

7) Don't include back-ticks around table names or columns names in the SQL query